



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

7

Concepts avancés

Sofiane HACHICHA
ISIMG 2025 - 2026

7- Concepts avancés

7.1 Classes scellées (1/3)

POO : Java



- Les classes scellées (sealed classes) ont été introduites pour la première fois en mode preview dans Java 15, puis proposées à nouveau en préversion dans Java 16. Elles sont devenues une **fonctionnalité standard et finalisée** à partir de **Java 17**.
- Une classe scellée, comme une interface scellée, permet de restreindre quelles classes peuvent l'étendre et quelles classes ou interfaces peuvent l'implémenter ou l'étendre, sans interdire totalement l'héritage.
 - Une classe scellée, comme une interface scellée, restreint les types autorisés à l'étendre ou à l'implémenter, sans supprimer totalement l'héritage.
- Pour définir une classe ou une interface scellée, on utilise le mot-clé **sealed**, suivi du mot-clé **permits** pour indiquer les types autorisés à en dériver ou à l'implémenter.
- Par exemple, si l'on veut limiter les figures géométriques, seules les classes Cercle et Rectangle seront autorisées de dériver de la classe Shape.



7- Concepts avancés

7.1 Classes scellées (2/3)



```
public sealed class Shape permits Cercle, Rectangle { - - - }
```

- Dans une hiérarchie scellée, chaque type autorisé (classe ou interface) doit préciser la manière dont il propage ou non la restriction d'héritage.
- Modificateurs autorisés sur les types dérivés :
 - **final** : La hiérarchie s'arrête à ce niveau. Aucun type ne peut hériter de cette classe.
 - **sealed** : Le type est lui-même scellé et doit spécifier, via une clause **permits**, quels types sont autorisés à l'étendre ou l'implémenter.
 - **non-sealed** : L'héritage redevient ouvert à partir de ce type : toute classe ou interface peut désormais l'étendre ou l'implémenter librement.
 - Cas particulier concernant les records :
 - Un record ne peut pas étendre une classe, qu'elle soit scellée ou non. Un record peut uniquement implémenter une interface.





7- Concepts avancés

7.1 Classes scellées (3/3)

- Ainsi, un record **ne peut être utilisé comme type dérivé que lorsqu'il implémente une interface scellée**, mais jamais pour hériter d'une classe scellée.
- Les types autorisés dans une hiérarchie scellée doivent être déclarés **final, record, sealed ou non-sealed**

```
final class Cercle extends Shape {- - - }
```

```
non-sealed class Rectangle extends Shape {- - - }
```

```
class ColoredRectangle extends Rectangle {- - - }
```



7- Concepts avancés

7.2 Pattern matching instanceof (1/4)

P.O.O : Java



Sofiane HACHICHA
ISIMG 2025 - 2026

● Le pattern matching pour instanceof a été introduit en Java 14 en mode preview, puis finalisé en Java 16.

- Il permet de vérifier le type d'un objet tout en liant automatiquement une variable de ce type, supprimant ainsi la nécessité d'un **cast explicite** et d'une déclaration séparée. Cette opération, autrefois réalisée en deux étapes, peut désormais être exprimée en une seule.

```
Object obj = "abc";
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}
```

```
if (obj instanceof String s) {
    // Utilisation de s, variable de motif ou pattern,
    // directement comme un String
    System.out.println(s.length());
}
```



7- Concepts avancés

7.2 Pattern matching instanceof (2/4)

P.O.O : Java



- Avec le pattern matching, il devient plus facile d'extraire et de manipuler les composants d'un record.

```
record Point(double x, double y) {}  
public class Test{  
    public static void print1(Object obj) {  
        if (obj instanceof Point p) {  
            double x = p.x();  
            double y = p.y();  
            System.out.println("X: " + x + ", Y: " + y);  
        }  
    }  
    public static void print2(Object obj) {  
        if (obj instanceof Point(double x, double y)) { // Depuis Java 21  
            System.out.println("X: " + x + ", Y: " + y);  
        }  
    }  
}
```



7- Concepts avancés

7.2 Pattern matching instanceof (3/4)

- Le pattern matching peut être utilisé pour travailler avec des classes et des records.

```
sealed interface Shape permits Circle, Rect {}
final class Circle implements Shape {
    double r;
    Circle(double r) { this.r = r; } }
final class Rect implements Shape {
    double len, w;
    Rect(double len, double w) { this.len = len; this.w = w; } }
public class TestShape {
    public static void printShapeInfo(Shape f) {
        if (f instanceof Circle c) {
            System.out.println("Cercle avec rayon: " + c.r);
        } else if (f instanceof Rect r) {
            System.out.println("Rectangle avec long:" + r.len + ",larg:" + r.w); } }
    public static void main(String[] args) {
        Shape obj = new Circle(2.5);
        printShapeInfo(obj); } }
```

Résultat d'affichage

POO : Java





7- Concepts avancés

7.2 Pattern matching instanceof (4/4)

POO : Java



```
sealed interface Forme permits Cercle, Rectangle {}
record Cercle(double r) implements Forme {}
record Rectangle (double lon, double larg) implements Forme {}
public class TestForme {
    public static void printFormeInfo(Forme f) {
        if (f instanceof Cercle(var r)) {
            System.out.println("Cercle avec rayon: " +r);
        } else if (f instanceof Rectangle(var l1, var l2)) {
            System.out.println("Rectangle avec long: " +l1 + ", larg: " +l2);
        }
    }
    public static void main(String[] args) {
        Forme obj = new Cercle(2.5);
        printFormeInfo(obj);
    }
}
```

Résultat d'affichage



7- Concepts avancés

7.3 Classes anonymes (1/4)

POO : Java



- La syntaxe, utilisée pour l'héritage de classe/mise en œuvre d'interface, est trop verbeuse pour des codes simples.

```
interface AfficheI {
    void affiche(String str);
}
class monAffiche implements AfficheI {
    public void affiche(String str){
        System.out.println(str);
    }
}
public class Test {
    public static void main(String[] args) {
        AfficheI obj = new monAffiche();
        obj.affiche("CPI2");
    }
}
```

Nécessite une nouvelle classe, donc un fichier, le tout pour allouer une **unique instance** et peu de lignes de code.

Résultat d'affichage





7- Concepts avancés

7.3 Classes anonymes (2/4)

● **But** : simplifier la syntaxe, utilisée pour l'héritage de classe ou la mise en œuvre d'interface, pour les codes simples

- Allocation d'une unique instance de la classe dans le code.
- Peu de méthodes et de lignes de code dans la classe.

● **Principes** :

- Omettre le nom de la classe.
- Donner le code de la mise en œuvre au moment de l'allocation.



7- Concepts avancés

7.3 Classes anonymes (3/4)

POO : Java



Sofiane HACHICHA
ISIMG 2025 - 2026

```
interface AfficheI {
    void affiche(String str);
}

/* class monAffiche implements AfficheI{
    public void affiche(String str){
        System.out.println(str);
    }
}

public class Test {
    public static void main(String[] args) {
        AfficheI obj = new monAffiche();
        obj.affiche("CPI2");
    }
} */

public class Test {
    public static void main(String[] args) {
        AfficheI obj = new AfficheI() {
            public void affiche(String str){
                System.out.println(str);
            }
        };
        obj.affiche("CPI2");
    }
}
```

Résultat d'affichage

Définit une nouvelle classe qui implémente (ou hérite) de **AfficheI** et qui n'a pas de nom.

Mise en œuvre au moment de l'allocation.

7- Concepts avancés

7.3 Classes anonymes (4/4)

P.O.O : Java



● Accès aux champs de la classe anonyme, mais aussi de la classe englobante et aux variables de la méthode englobante

● Accès en **lecture seule** aux variables de la méthode

```
interface OperationI {
    int calcul(); }
public class Somme {
    int x=1;          /* accès en lecture/écriture */
    public void f() {
        int y=2;      /* accès en lecture seule */
        OperationI o = new OperationI() {
            int z;      /* accès en lecture/écriture */
            public int calcul(){
                z=x+y;
                return z;
            }
        };
        System.out.println(o.calcul());
    }
    public static void main(String[] args) { new Somme().f(); }
}
```

Résultat d'affichage



7- Concepts avancés

7.4 Classes internes (1/5)

P.O.O : Java



- Une classe interne est une classe définie dans une autre classe

- Permet de **coupler** fortement deux classes (la classe interne accède aux champs de la classe externe).

```
class Englobante {
    private int b;
    class Englobee {
        int a;
        void f() { a = 1; b = 2; }
    }
}
```

- Une classe interne possède un champ ajouté automatiquement permettant d'accéder à la classe externe.

```
void f() { a = 1; b = 2; }
```



```
void f() { this.a = 1; Englobante.this.b = 2; }
```



7- Concepts avancés

7.4 Classes internes (2/5)

P.O.O : Java



- À partir de la classe externe, l'allocation est simple :

```
class Englobante {  
    Englobe e;  
    class Englobe { ... }  
    Englobante() {  
        e = new Englobe();  
    }  
}
```

- À partir de l'extérieur de la classe englobante :

```
Englobante x = new Englobante();  
Englobante.Englobe y = x.new Englobe();  
x.e = y;          /* à faire dans main */
```

- Les champs de la classe englobée sont toujours accessibles à partir de la classe englobante et vice-versa.



7- Concepts avancés

7.4 Classes internes (3/5)



● Composition des opérateurs de visibilité pour l'extérieur

```
public class Englobante { /* accessible partout */
    private int b;          /* accessible dans Englobee */
    private Englobee e = new Englobee(); /* idem */
    class Englobee {        /* accessible dans le package */
        private int x;      /* accessible dans Englobante */
    }
}
```

● Classe interne de méthode = classe définie dans une méthode

- La classe interne peut accéder aux variables locales de la méthode (accès en lecture après la déclaration).

```
class Englobante {
    int x;
    void f() {
        int y = 2; /* y en lecture seule dans Englobee */
        class Englobee { void g() { x = y; } };
        Englobee e = new Englobee();
        e.g();
    }
}
```



7- Concepts avancés

7.4 Classes internes (4/5)

P.O.O : Java



● **Classe anonyme = classe interne de méthode sans nom**

```
void f() { /* en version nommée */  
    class AvecNom implements OperationI {  
        public int calcul() { return 1;}  
    };  
    OperationI x = new AvecNom();  
}
```



```
void f() { /* en version anonyme */  
    OperationI x = new OperationI() {  
        public int calcul() { return 1;}  
    };  
}
```



7- Concepts avancés

7.4 Classes internes (5/5)

P.O.O : Java



● Si la classe interne est marquée **static**, il s'agit d'une classe interne statique :

- Elle n'est pas liée à une instance de la classe englobante.
- Elle ne peut pas accéder directement aux champs d'instance de la classe englobante.
- Elle peut être instanciée sans créer d'instance de la classe englobante.

```
class A {  
    private int b;  
    private static int c;  
    static class B {  
        int a;  
        void f() { a = 1; c = 2; } /* pas d'accès direct à b */  
    }  
}  
/* allocation avec A.B x = new A.B(); */
```





7- Concepts avancés

7.5 Expressions Lambda (1/6)

- Souvent, le code de la mise en œuvre d'interface reste verbeux, même avec les classes anonymes.
- **Une expression lambda** simplifie la mise en œuvre d'une **interface fonctionnelle**. Elle permet d'écrire du code plus concis pour la méthode abstraite de cette interface.
 - la définition d'une expression lambda se fait sans déclaration explicite du type de retour, des types de paramètres, de modificateur de visibilité ou du nom de la méthode.
- Lorsque l'expression lambda est évaluée par le compilateur, celui-ci **infère** le type vers l'interface fonctionnelle.
 - Cela lui permet d'obtenir des informations sur **les paramètres utilisés, le type de retour**, etc.
- La syntaxe d'une expression Lambda est la suivante :
(listeDesParamètres) -> corps;
 - L'opérateur **->** sépare le ou les paramètres du bloc de code qui va les utiliser.

7- Concepts avancés

7.5 Expressions Lambda (2/6)

POO : Java



```
interface AfficheI {
    void affiche(String str);
}

public class Test {
    public static void main(String[] args) {
        /* AfficheI obj = new AfficheI() {
            public void affiche(String str){
                System.out.println(str);
            }
        }; */
        AfficheI obj = (str) -> { System.out.println(str); };
        obj.affiche("CPI2");
    }
}
```

On ne garde que la
partie « intéressante »
de la déclaration.

- **listeDesParamètres** : liste d'arguments déclarés à gauche de l'opérateur -> , avec un éventuel type, entre parenthèses; si un seul argument est présent, on peut omettre les parenthèses.
- **corps** : le corps d'une expression lambda est déclaré à droite de l'opérateur ->. Il contient :





7- Concepts avancés

7.5 Expressions Lambda (3/6)

- soit une seule instruction ; sa valeur, si elle existe, est renvoyée.
- soit un bloc d'instructions comportant plusieurs instructions suivies par un **return** et entourées par des accolades.
- il faut terminer une expression Lambda par un ";"
- Les **paramètres d'une expression Lambda** doivent correspondre à ceux définis dans **la méthode abstraite de l'interface fonctionnelle**.
- Les paramètres d'une expression lambda doivent respecter certaines règles :
 - une expression lambda peut n'avoir aucun, un seul ou plusieurs paramètres.
 - des parenthèses vides indiquent qu'il n'y a pas de paramètre.
 - le type des paramètres peuvent être explicitement déclaré ou être inféré par le compilateur selon le contexte dans lequel l'expression est utilisée.



7- Concepts avancés

7.5 Expressions Lambda (4/6)

- les paramètres sont entourés de parenthèses, chacun étant séparé par une virgule. S'il n'y a qu'un seul paramètre dont le type n'est pas explicitement précisé, alors l'utilisation des parenthèses n'est pas obligatoire.
- Le corps d'une expression Lambda doit respecter certaines règles:
 - il peut n'avoir aucune, une seule ou plusieurs instructions.
 - lorsqu'il ne contient qu'une seule instruction, les accolades ne sont pas obligatoires et la valeur de retour est celle de l'instruction si elle en possède une.
 - lorsqu'il y a plusieurs instructions, elles doivent obligatoirement être entourées par des accolades.
 - la valeur de retour est celle de la dernière expression ou void si rien n'est retourné.



7- Concepts avancés

7.5 Expressions Lambda (5/6)

P.O.O : Java



Exemples :

Expressions Lambda	Description
() -> { System.out.print("Bonjour"); }	Afficher "Bonjour" sur la sortie standard.
() -> 10; () -> { return 10; }	N'accepter aucun paramètre et renvoyer la valeur 10.
x -> x * 2;	Accepter un nombre et renvoyer son double.
(x, y) -> x * y; (int x, int y) -> x * y;	Accepter deux nombres et renvoyer leur produit
(String s) -> System.out.print(s); (s)-> System.out.print(s); s-> System.out.print(s);	Accepter une chaîne de caractères et l'afficher sur la sortie standard.



7- Concepts avancés

7.5 Expressions Lambda (6/6)

P.O.O : Java



Expressions Lambda	Description
(Point p, float x) -> {System.out.println("Valeur="+x); p.afficher();};	Accepter deux paramètres et les afficher sur la sortie standard sans rien renvoyer.
(val1, val2) -> { return val1 >= val2; }; (val1, val2) -> val1 >= val2;	Renvoyer un booléen qui précise si la première valeur est supérieure ou égale à la seconde.
() -> { for (int i = 0; i < 10; i++) traiter(); }	Exécuter la méthode traiter() dix fois.
p -> p.getSexe() == Sexe.HOMME && p.getAge() >= 10 && p.getAge() <= 80	Renvoyer un booléen pour un objet possédant une méthode getSexe() et getAge() qui vaut true si le sexe est masculin et l'age compris entre 10 et 80 ans

7- Concepts avancés

7.6 Références de méthodes (1/4)



- Une **référence de méthode** est un moyen qui permet d'offrir une syntaxe simplifiée pour **invoquer une méthode comme une expression lambda**, sans fournir une implémentation.
 - Elle fournit un **raccourci syntaxique** pour créer une expression lambda dont le but est d'invoquer une méthode statique ou d'instance ou un constructeur.
- Les références de méthodes utilisent l'opérateur **::**
- La syntaxe d'une référence de méthode est composée de trois éléments :
 - un **qualificateur** qui précise le nom d'une classe ou d'une instance sur lequel la méthode sera invoquée
 - l'opérateur **::**
 - un **identifiant** qui précise le nom de la méthode ou l'opérateur new pour désigner un constructeur



7- Concepts avancés

7.6 Références de méthodes (2/4)

● Il existe **quatre types** de références de méthodes :

Type	Syntaxe	Exemple
Référence à une méthode statique	nomClass::nomMethodStatic	Integer::valueOf
Référence à une méthode d'instance	objet::nomMethode	personne::toString
Référence à une méthode d'un objet arbitraire d'un type donné	nomClasse::nomMethode	String::concat
Référence à un constructeur	nomClasse::new	Personne::new

P.O.O : Java



7- Concepts avancés

7.6 Références de méthodes (3/4)

- Le tableau ci-dessous présente des exemples de références de méthodes et leurs expressions lambda équivalentes :

Type	Référence de méthode	Expression Lambda
Référence à une méthode statique	System.out::println	x -> System.out.println(x);
Référence à une méthode d'instance	monObj::maMethod	x -> monObj.maMethod(x);
Référence à une méthode d'un objet arbitraire d'un type donné	String::toUpperCase	(String s) -> s.toUpperCase();
Référence à un constructeur	MaClasse::new	str -> new MaClasse(str);





7- Concepts avancés

7.6 Références de méthodes (4/4)

P.O.O : Java



```
interface AI{ Integer test(int num); }
interface BI{ String test(); }
interface CI{ Person test(String str); }
interface DI{ String test(String str1, String str2); }
class Person{
    private String nom;
    public Person(String nom){ this.nom=nom; }
    public String toString() {return nom;}
}
public class TestRef {
    public static void main(String[] args) {
        Person p = new Person("ali");
        AI expr = Integer::valueOf;
        System.out.println(expr.test(10));
        BI p1ref = p::toString;
        CI p2ref = Person::new;
        System.out.println(p1ref.test());
        System.out.println(p2ref.test("salma"));
        DI p3ref = (s1,s2)->s1.concat(s2);
        DI p4ref = String::concat;
        System.out.println(p3ref.test("abc","def"));
        System.out.println(p4ref.test("abc","def"));
    }
}
```

Résultat d'affichage

